SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

RefundFlow AI is a GLM-powered workflow engine designed to convert unstructured refund requests into structured refund cases. It detects the platform and refund reason, checks missing information, generates platform-specific workflow tasks, prepares a refund claim package, and tracks the case status until the next action is clear.

1.1. Purpose:

The main goal of RefundFlow AI is to guide users through the refund process across different buying platforms by turning unstructured messages into platform-ready refund workflows. The system analysis document highlights the technical scope and design-related decisions behind its multi-agent automated workflow development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
    a. *RefundFlow AI is a cloud-based platform. The backend operates on a FastAPI framework and functions using a multi-agent task dispatch architecture. The system operates as a stateful refund workflow engine.*
2. ***Data Flows:***
    a. *Users enter a refund problem in natural language via the frontend, optionally providing structured fields such as platform, order ID, item name, purchase amount, and evidence description. The frontend sends this request to the backend API endpoint (POST /api/refunds/analyze). The backend validates the input, builds the context, and sends it to the GLM reasoning layer, which identifies the intent, refund reason, missing information, workflow stage, and next action. Finally, the backend parses this structured output to create a refund case, and the frontend displays the tasks, missing fields, channel recommendations, and an audit trail for the user.*
3. ***Model Process:***
    a. *The model workflow utilizes a multi-step agentic prompting strategy. Instead of asking the GLM for a single answer, the backend prompts the GLM to perform ordered reasoning steps: detecting the intent, classifying the refund reason, identifying missing fields, selecting the workflow stage, generating tasks, and returning a structured JSON output.*
4. ***Role of Reference:***
    a. *This document serves as a reference for all developers and testers to enable the overall high-level architecture of this project.*

© UMHackathon 2026

Email: umhackathon@um.edu.my

1.2. ***Background****:*

Consumers frequently purchase from various online platforms, leading to fragmented refund procedures. Users often provide messy text (chat messages, incomplete details, emotional complaints), causing repeated back-and-forth communication, missed evidence, incorrect channel selection, and delayed outcomes. For support teams, this manual process of identifying platform, order ID, reason, and next step is repetitive.

1. *Previous Version:*
    a. *Currently, refund processes are siloed within each individual platform (e.g., Shopee, Lazada, Amazon). Users must manually figure out each platform's specific evidence requirements, support channels, and return windows. Customer support agents or sellers must manually read messy, unstructured messages to identify the platform, product, refund reason, and necessary evidence before taking any action.*
2. *Changes in Major Architectural Components*
    a. *The system architecture shifts from traditional, rigid form-based data entry and manual ticketing systems to an AI-driven Multi-Agent Task Dispatch architecture. It introduces a central reasoning layer powered by the Z.AI GLM integrated with a FastAPI backend, enabling intelligent, stateful workflow tracking rather than relying on static, manual database updates.*
3. *New capabilities that are only introduced in this*
    a. *The introduction of a GLM-powered workflow engine capable of parsing unstructured natural language into structured, platform-ready refund cases. New capabilities include automated refund reason classification, proactive missing information detection (e.g., identifying missing order IDs or specific evidence), and dynamic, platform-specific task orchestration based on the user's detected intent.*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Online Shoppers	*Submits refund complaints and unstructured order details via the frontend interface.*	*Simple and clear guidance on missing information, step-by-step tasks to resolve their issue, and faster preparation of their refund request.*
Customer Support Agents	*Reviews escalated refund cases and processes them based on platform policies.*	*To receive structured, categorized case information and required evidence upfront instead of reading messy, long messages.*
Small Business Sellers	*Manages returns, refunds, and disputes for their specific store products.*	*Faster triage of return/refund issues, enabling quicker resolution of customer disputes.*
Marketplace Operations Teams	*Oversees the overall refund ecosystem, support channels, and dispute resolutions.*	*Consistent refund categorization, standardized evidence collection, and clear task tracking across the platform.*
Development Team	*Builds the multi-agent architecture, integrates the Z.AI GLM API, and maintains the FastAPI backend.*	*Clear API usage contracts, a modular codebase, and a well-documented prompt engineering architecture.*
Hackathon Judges / Demo Users	*Evaluates the RefundFlow AI prototype during UMHackathon 2026.*	*To see a functional, clear AI workflow engine demonstrating the integration of the backend, frontend, and GLM, alongside testing evidence.*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web-based Frontend & RESTful API Backend*
Architecture	*Multi-Agent Task Dispatch Architecture (Cloud-ready)*

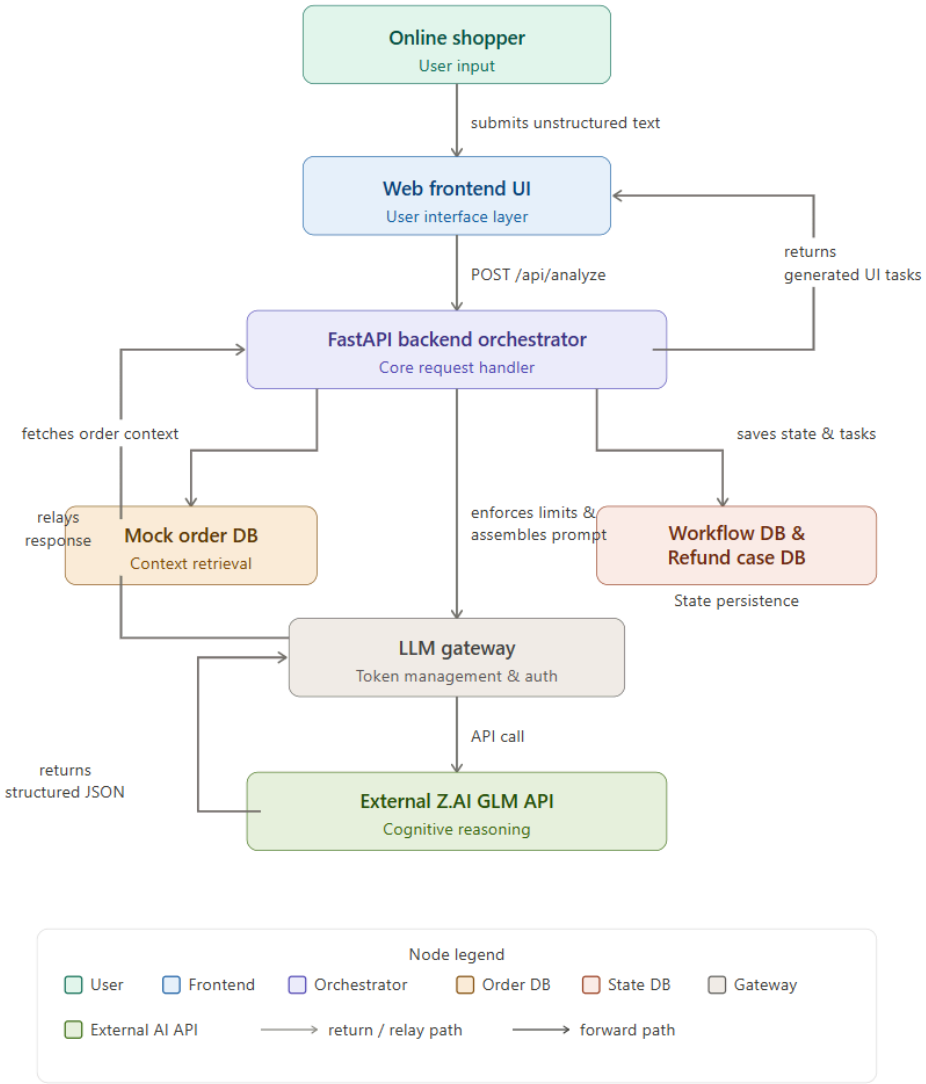
RefundFlow AI is structured as a client-server-based system utilizing a centralized intelligence layer. The client is a Web-based Frontend UI where users submit their unstructured queries. This client communicates via RESTful HTTP requests to the FastAPI Backend layer. The backend operates as the orchestrator—housing the multi-agent task dispatch logic—and communicates with independent mock databases (workflow_db, refund_case_db, and mock_order_db). For cognitive processing, the backend acts as an API gateway that securely communicates with the external Z.AI GLM-4.5 API. While currently utilizing in-memory databases for the prototype, the architecture is designed to be fully deployable as microservices on cloud infrastructure.

2.1.2 LLM as Service Layer

The architecture integrates the Z.AI GLM not just as a generic add-on, but as the core reasoning engine that dictates state changes in the workflow.

© UMHackathon 2026

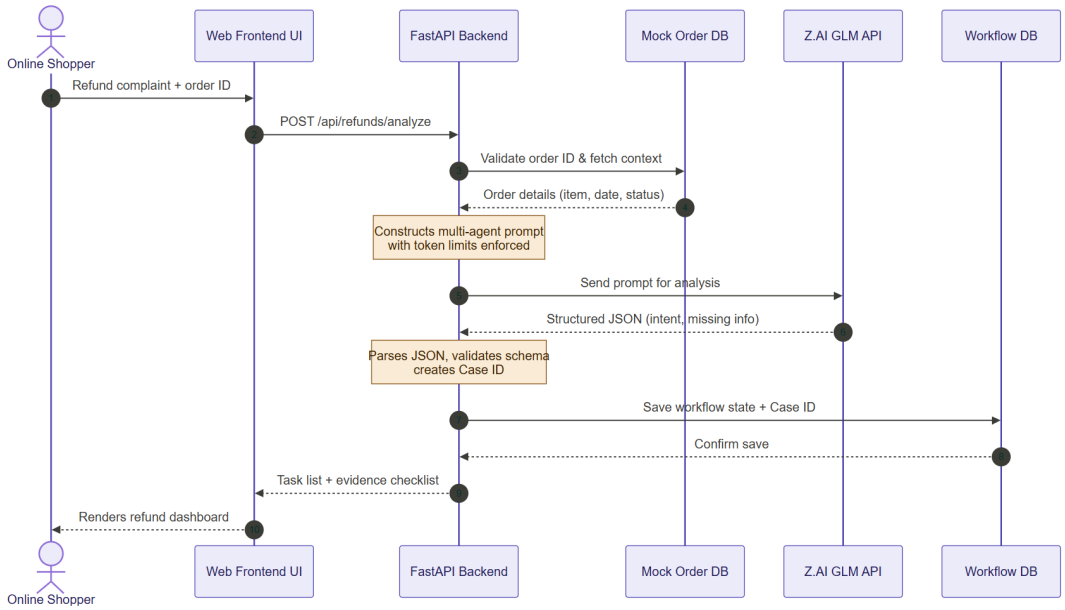
2.1.3 Dependency Diagram



© UMHackathon 2026

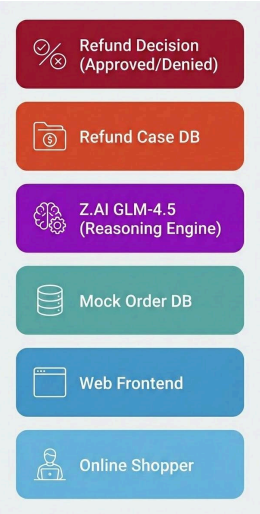
Email: umhackathon@um.edu.my

2.2.2. Sequence Diagram



a.

2.2. Technological Stack

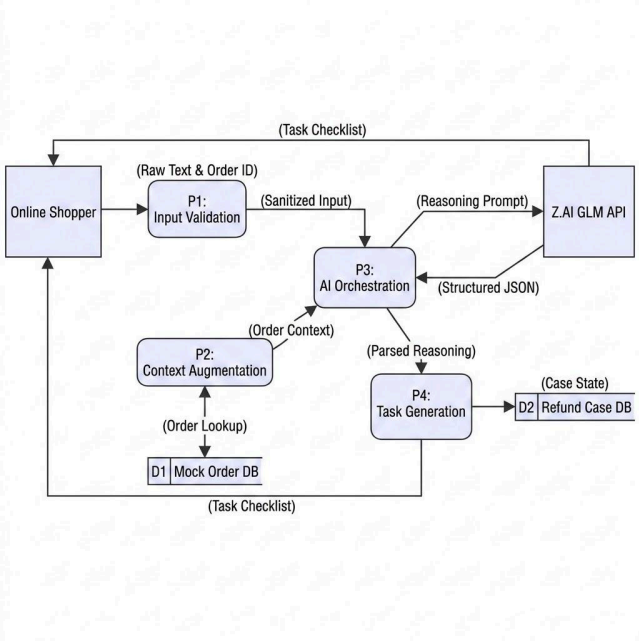


© UMHackathon 2026

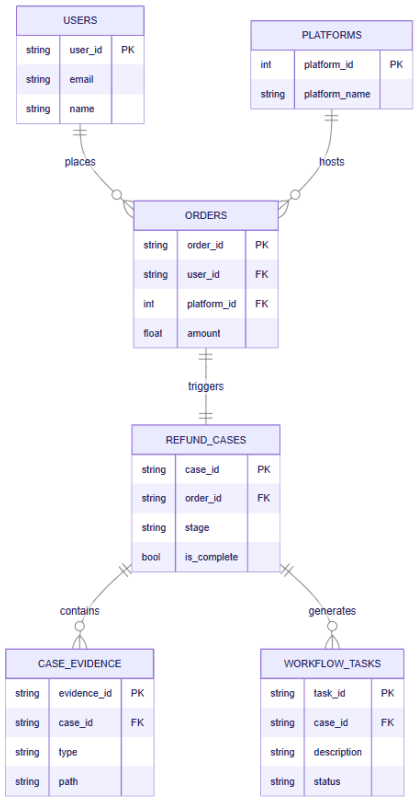
Email: umhackathon@um.edu.my

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored



2.3.2. Normalized Database Schema



© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

	Feature	Description
1	**Unstructured Refund Intake**	*The system accepts messy, unstructured natural language input (e.g., chat logs, emotional complaints) alongside optional structured fields (Order ID, Platform) via a Web UI and passes it to the FastAPI backend.*
2	**Refund Reason Classification**	*The Z.AI GLM engine evaluates the input to automatically classify the refund intent (e.g., Missing Parcel, Damaged Item, Wrong Item, Late Delivery).*
3	**Missing Information Detection**	*The system proactively flags essential missing data required for a specific platform's claim (e.g., missing photographic evidence, courier tracking number, or item value) before proceeding.*
4	**Dynamic Task Orchestration**	*Based on the classified reason and platform-specific adapter rules, the backend orchestrates the `next_action` (e.g., `ask_follow_up_questions`, `generate_refund_checklist`, `create_refund_case`).*
5	**Stateful Case Tracking**	*Generates a unique Refund Case ID, storing the extracted JSON payload, current workflow stage, and recommended user actions into the system's mock databases (`workflow_db`, `refund_case_db`).*

© UMHackathon 2026

3.1. Minimum Viable Product:

3.1.1. *Do list down 3-5 core features that the team is going to build and do a demo by the end of the UMHackathon 2026.*

#	Features	Description
1	Unstructured Refund Intake	The system accepts messy, unstructured natural language input (e.g., chat logs, emotional complaints) alongside optional structured fields via a Web UI and passes it to the FastAPI backend.
2	Refund Reason Classification	The backend queries the Z.AI GLM engine to automatically evaluate the input and classify the refund intent (e.g., Missing Parcel, Damaged Item, Wrong Item, Late Delivery).
3	Missing Information Detection	The workflow proactively flags essential missing data required for a specific platform's claim (e.g., missing photographic evidence, courier tracking number) before proceeding.
4	Dynamic Task Orchestration	Based on the classified reason and platform-specific adapter rules, the system orchestrates the next_action (e.g., ask_follow_up_questions, generate_refund_checklist, create_refund_case).
5	Stateful Case Tracking	Generates a unique Refund Case ID, storing the extracted JSON payload, current workflow stage, and recommended user actions into the system's mock databases.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*The system must handle multiple concurrent refund analysis requests efficiently during the hackathon demonstration without blocking.*	*Use asynchronous routing (async def) in the FastAPI backend to ensure concurrent processing during the Z.AI API network calls.*
Reliability	*The reasoning engine must consistently output parsable data to prevent workflow crashes or orphaned request states.*	*Implement strict JSON-schema prompting and a backend fallback mechanism that automatically retries once if the GLM outputs malformed, non-JSON text.*
Data Privacy	*The prototype must prevent the permanent storage of real Personal Identifiable Information (PII) or sensitive payment credentials.*	*Utilize transient in-memory mock databases (workflow_db, refund_case_db) that reset upon server restart, and prompt users not to submit real sensitive data.*
Token Latency	*The API calls to the Z.AI GLM must process quickly enough to prevent frontend client timeouts.*	*Implement a maximum input limit (approx. 6,000 characters) via truncation to ensure optimal token processing speed, and display dynamic loading states on the frontend.*
Cost Efficiency	*Average token consumption per user session must remain low to manage GLM inference costs at scale.*	*Only inject minimal, necessary platform-specific validation rules (adapters) into the context window rather than passing full, exhaustive platform policy documents.*

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

This section identifies the high-level features that the team planned for the "Grand Vision" of RefundFlow AI, but which are not part of the MVP scope for UMHackathon 2026.

a. *Real E-Commerce API Integration: Direct API integration to automatically submit the refund claims to actual platforms (e.g., Shopee, Lazada, TikTok Shop, Amazon) or bank dispute systems. The current prototype stops at preparing the claim package and generating tasks.*
b. *Multimodal Image Verification: Uploading and automated AI analysis of photographic evidence (e.g., the system directly analyzing an uploaded screenshot to verify if it clearly shows a "damaged item"). The current MVP relies on users providing a text description of their evidence.*
c. *Automated Account Scraping: Direct scraping of user e-commerce accounts or automated login sessions to automatically retrieve order histories and tracking statuses.*

d. *Automated Final Approval: AI making the final decision to approve a refund and issue money. This will remain out of scope permanently, as final approval depends strictly on the respective platform's policies and human review stages.*

e. *Persistent Storage of PII: Long-term database storage of real Personal Identifiable Information (PII) or sensitive payment credentials. The prototype strictly utilizes transient mock data for demonstrations.*

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

4.1.1. ***Grayscale Rollout & A/B Testing****: New prompt strategies and platform-specific adapter rules will initially be released to a smaller subset (e.g., 10%) of test users/demo cases. The system will monitor the JSON parsing success rate and accuracy of refund reason classification. If no issues are detected, the update is rolled out to 100%. A/B testing will be conducted to compare single-prompt versus multi-step agentic prompting to determine which variant yields higher structural accuracy and lower token consumption.*

4.1.2. ***Emergency Rollback (ER) & Golden Release****: In terms of recovery and traffic management, if there are sudden anomalies—such as the Z.AI GLM consistently returning malformed, non-JSON text or timing out—an Emergency Rollback (ER) is triggered. The system will automatically route requests to the Golden Release (a deterministic "mock mode" logic) ensuring users still receive basic, static refund guidance rather than a system crash.*

© UMHackathon 2026

4.1.3. Priority Matrix:

Priority Level	*Trigger Conditions*	*Actions Taken*
P1 Critical	*GLM API failure, timeout, or complete JSON schema parsing failure error rate exceeds 5% of traffic.*	*Immediate emergency rollback to the deterministic "mock mode" Golden release; graceful error state displayed to users.*
P2 High	*In-memory mock database (workflow_db or refund_case_db) fails to read/write state updates.*	*Alert development team; reject new sessions with an HTTP 500 error preventing orphaned workflow states.*
P3 Medium	*Z.AI API response latency exceeds 5 seconds under normal load.*	*Check token truncation/chunking logic; ensure frontend loading UI is extended to manage user expectations.*

4.2. ***Monitoring****:*

4.2.1. The monitoring plan focuses on the health of the FastAPI backend and the external LLM integration. The system will actively track endpoint latency (specifically POST /api/refunds/analyze), average token consumption per session, and JSON validation success rates. If the failure rate for JSON parsing crosses the defined threshold, or if API limits are hit, automated warnings will trigger to shift the system into fallback mode.

© UMHackathon 2026

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.*

- ***a.*** *Users have stable internet connections on their devices to submit unstructured text inputs and receive real-time workflow updates.*
- ***b.*** *Current e-commerce platform refund policies (e.g., Shopee, Lazada) remain relatively static during the prototype phase, allowing the designed "platform adapter rules" to be valid.*
- *c. The Z.AI GLM-4.5 engine will consistently adhere to system instructions requesting strictly formatted JSON output.*

4.4. **External Dependencies:** *Do list all the external tools that are applied to your system to function. [e.g. below]*

Tools	*Purpose*	*Risks*
Z Ai GLM API	*It acts as the core LLM reasoning engine handling prompt inference, intent classification, multi-step reasoning, and generating the structured JSON workflow output.*	*High - In case of GLM API rate limitation or downtime, the core product will fail. Mitigation: Implement a graceful degradation state (mock mode) and strict schema fallback logic.*
Z.AI GLM API (Text Extraction)	*Parsing user natural language to identify missing order fields (Order ID, Item Name, Evidence).*	*Medium - Risk of hallucinating data not present in the user text. Mitigation: Enforce strict prompt boundaries and cross-reference with the mock database.*
FastAPI Framework	*Serving as the asynchronous backend orchestrator to route traffic and assemble multi-agent prompts.*	*Low - Highly stable, but potential for unhandled exceptions if async tasks block. Mitigation: Comprehensive exception handling and routing rules.*

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

© UMHackathon 2026

5.1. *Project Timeline:*

Timeline	*Development Phase*	*Justification / Tasks Completed*
Day 1-2	*Project Planning & Architecture*	*Brainstormed the core problem, finalised the Product Requirement Document (PRD), designed the Multi-Agent Task Dispatch architecture, and mapped out the JSON schema for the GLM outputs.*
Day 3-4	*Backend Setup & UI Scaffolding*	*Initialised the FastAPI backend, set up the in-memory mock databases (workflow_db, mock_order_db), and built the foundational Frontend Web UI components.*
Day 5-6	*AI Integration & Prompt Engineering*	*Connected the Z.AI GLM-4.5 API. Designed and refined the multi-step agentic prompting strategy and platform-specific adapter rules to ensure strict, parsable JSON outputs.*
Day 7-8	*System Integration*	*Connected the Web Frontend to the FastAPI backend. Ensured that unstructured user text flows correctly through the system and that generated tasks dynamically render on the UI dashboard.*
Day 9	*Quality Assurance & Edge Testing*	*Conducted testing as per the QATD. Performed adversarial testing (hallucination checks, missing data triggers) and tested the GLM fallback/retry mechanisms.*

© UMHackathon 2026

Day 10	***Final Polish & Submission Prep***	***Finalised the System Analysis Documentation (SAD), recorded the prototype demonstration/pitch video, and performed a final code cleanup for the GitHub repository submission.***

5.2. ***Team Members Role:*** *Highlight the role of team members*

Tan Kwan Hong – Role AI & Backend Lead – GLM API Integration, Prompt Design & Structured Output

Kong Yih Liang – Role Workflow Engine Lead – Step Execution, State Management & Tool Functions

Joshua Mugan Bunyau – Role Frontend Developer – UI Implementation, Input/Result Display & Progress Tracking

Jeremy Hii Neng Yii – Role Product & Pitch Lead – PRD, QA Documentation, Pitch Slides & Script

5.3. Recommendations:

Performance (Caching): Implement a Redis Cache layer to store the context of recently analysed, highly similar refund complaints. This will drastically reduce redundant API calls to the Z.AI GLM, lowering token costs and improving response latency for common issues.

Scalability (Database & Async queues): Transition the transient in-memory databases (workflow_db) to a persistent relational database like PostgreSQL. Additionally, integrate a message broker (e.g., Celery / RabbitMQ) to handle the LLM API calls as background tasks, preventing the FastAPI endpoints from timing out during high concurrent traffic.

Reliability (Circuit Breakers): Implement a circuit breaker pattern in the backend. If the Z.AI API experiences prolonged latency or rate-limiting, the circuit breaker will automatically pause external requests and serve users a cached or deterministic fallback UI until the external system stabilizes.

© UMHackathon 2026